Week-5

01 November 2026 01:03

Unweighted graph:

Adj_list = {0:[1,2], 1:[2,5,6], 2:[3,4,6,1], 3:[1,5,4], 4:[], 5:[], 6:[4]}

Directed unweighted graph

Adj_mat = [[0,1,1,0,0,0,0],
[0,0,1,0,0,1,],
[0,1,0,1,1,0,1],
[0,1,0,0,1,1,0],
[0,0,0,0,0,0,0],
[0,0,0,0,0,0,0],
[0,0,0,0,1,0,0]]

For undirected graph, adjacency matrix is symmetric

Weighted graph:

Adj_list
= {0:[(1,w_1),(2,w_2)], ...}

Adj_mat = [[(0,0),(1,w_1),(1,w_2),(0,0),(0,0),(0,0),(0,0)],
. . . .
. . . .]

= [[-∞, w_1, w_2, -∞, -∞, -∞, -∞], # ALTERNATIVE REPRESENTATION
. . . .
. . . .]

Shortest Path Algorithms

⇒ Find shortest path from given source vertex to every other vertex

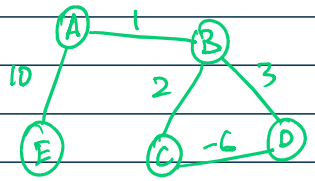
No -ve cycles, no -ve weights — Dijkstra
No -ve cycles, +ve/-ve weights — Bellmann-Ford

} a.k.a Single Source Shortest Path

Dijkstra will never work for all -ve weighted graphs (True/False) ✓

Dijkstra will not always work for -ve weighted graphs (True/False) ✓

Can we identify if a graph has -ve cycle using Bellmann Ford? (T/F) ✓

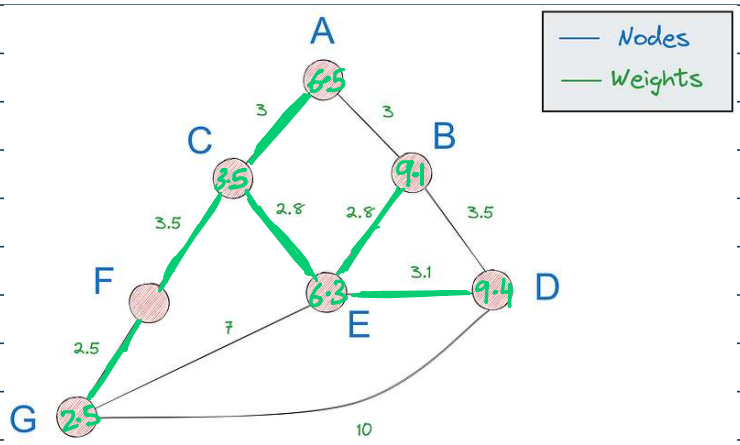


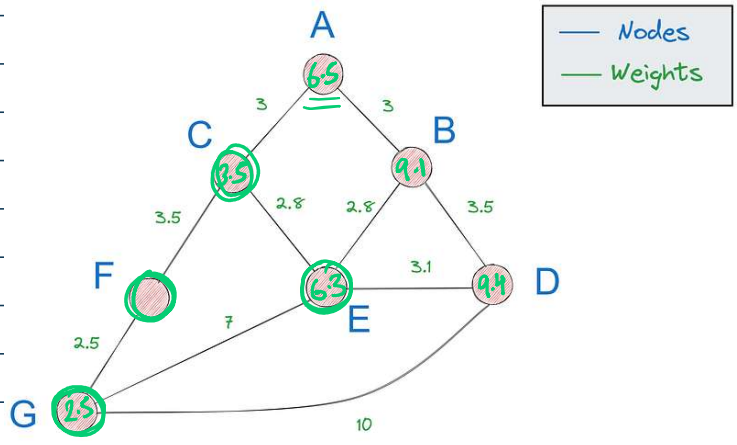
source vertex is B

If a graph has -ve cycle, then shortest path is undefined.

⇒ Find the shortest path from every vertex to every other vertex
No -ve cycle, +ve/-ve weights
Floyd-Warshall

DIJKSTRA(source_vertex = F)



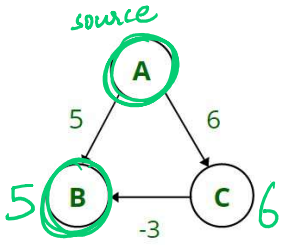


DIJKSTRA'S TIME COMPLEXITY

⇒ Considering optimal implementation

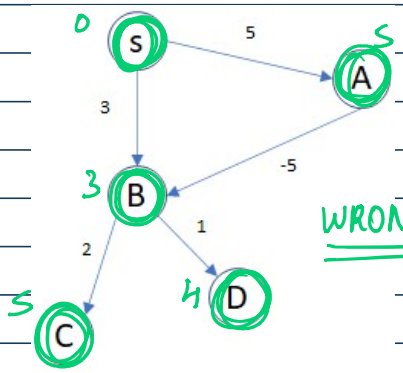
Adj_list $O((E+V)\log V)$
Adj_mat $O(V^2)$
} Using min-heap OR priority-queue (Will be discussed in upcoming sessions)

~~Adj_list $O(V^2+E) \approx O(V^2)$~~
Adj_mat $O(V^2)$
} ~~Naive implementation~~ Array/list based

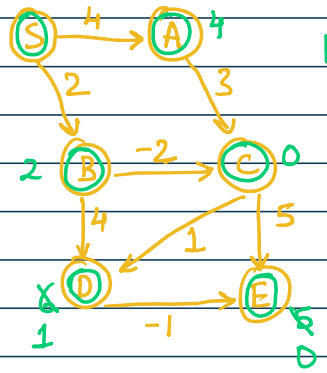


This problem exist because dijkstra is 'greedy', it marks nodes visited before actually finding shortest path (when -ve weight edges are involved)

edges are involved)

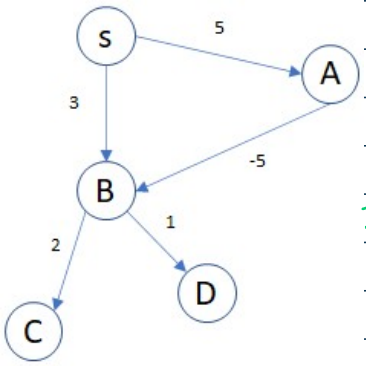


WRONG



Here Dijsktra works, BUT not always the case

BELLMANN-FORD (source_vertex = S)



S	A	B	C	D
Init 0	∞	∞	∞	∞

Explore edges from source in any order

Itr 1

Repeat

considering optimal implementation

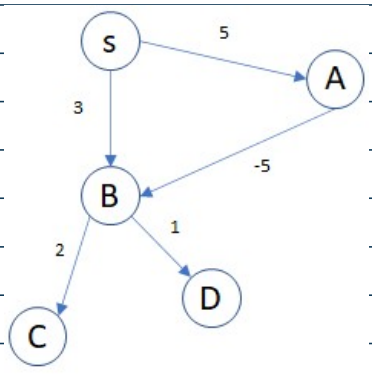
Adj_list
O(V·E)

Adj_mat
O(V3)

Repeat exploring edges from source |V|−1 times

And if at |V|th time, distance is updated ⇒ -ve cycle present

FLOYD-WARSHALL (All-pairs shortest path)



S A B C D

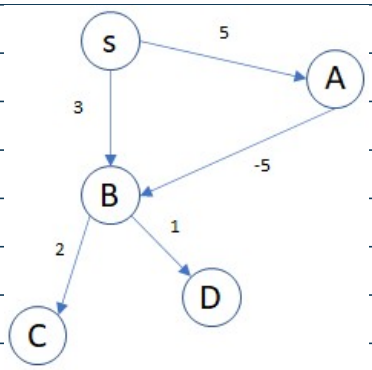
dist = [[∞,∞,∞,∞,∞], S
[∞,∞,∞,∞,∞], A
[∞,∞,∞,∞,∞], B
[∞,∞,∞,∞,∞], C
[∞,∞,∞,∞,∞]] D

Direct edges

∞, 5, 3, ∞, ∞
∞, ∞, -5, ∞, ∞
∞, ∞, ∞, 2, 1
∞, ∞, ∞, ∞, ∞
∞, ∞, ∞, ∞, ∞

dist[i][j] = min(dist[i][j], dist[i][k] +





∞, ∞, ∞, ∞, ∞

dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

	∞, 5, 0, 2, 1
intermediate nodes = 1	∞, ∞, -5, -3, -4
	∞, ∞, ∞, 2, 1
	∞, ∞, ∞, ∞, ∞
	∞, ∞, ∞, ∞, ∞